❸ You can create deeply nested structures to organize your configuration values better.

I've nested the new configuration inside the `MapSettings` parent key to create a *section* that will be useful later for binding values to a POCO object. I also nested the `latitude` and `longitude` keys under the `DefaultLocation` key. You can create any structure of values you like; the configuration provider will read them fine. Also, you can store the values as any data type—numbers, in this case—but be aware that the provider will read and store them internally as strings.

TIP The configuration keys are *not* case-sensitive in your app, so bear that fact in mind when loading from providers in which the keys are case-sensitive. If you have a YAML file with keys `name` and `NAME`, for example, only one will appear in the final `IConfiguration`.

Now that you have a configuration file, it's time for your app to load it into the `ConfigurationManager`.

10.2.1 Adding a configuration provider in Program.cs

As you've seen throughout this book, ASP.NET Core (from .NET 6 onward) uses the `WebApplicationBuilder` class to bootstrap your application. As part of the bootstrap process, `WebApplicationBuilder` creates a `ConfigurationManager` instance and exposes it as the property `Configuration`.

TIP You can access the `ConfigurationManager` directly on `WebApplicationBuilder .Configuration` and `WebApplication.Configuration`. Both properties reference the same `ConfigurationManager` instance.